

从零训 Code LLM · 阶段二

SFT、数据清洗的曲折,与一个迟到的教训

SFT 对齐 · 数据质量的反复踩坑 · 0.6B 小模型的天花板 · KV-cache/RoPE debug · 以及"从零训 vs 强 base"的路线修正

本篇脉络

- 0. 承上:阶段一(预训练会写 C++)到阶段二(SFT 教解题)
- 1. SFT 是什么:warm start、题解格式、只在答案算 loss
- 2. 变长样本组 batch:padding + 一个 loss mask 就够(causal 的天然隔离)
- 3. 数据清洗的反复踩坑:混 Python、编译太严、启发式的权衡
- 4. Raw 模型训完后的检查:token 覆盖诊断 / 单字节兜底 / embedding 稀疏更新验证 / 生成测试
- 5. KV-cache 的 RoPE 位置 bug:训练对、推理崩(附定量分析)
- 6. 验证:见过的题 vs 没见过的题——SFT 记忆,不泛化
- 7. 迟到的教训:0.6B 的天花板(业界最强 0.5B 才 HumanEval 17%)
- 8. 路线修正:从零训小模型 vs 现成强 base + post-train
- 9. 各阶段能力占比:预训练是地基,SFT/RL 是锦上添花

0. 承上:从"会写 C++"到"教解题"

阶段一从零(随机初始化)手写 Transformer,在 14B C++ token 上预训练出一个 0.6B 模型,会写通顺的 C++ (loss 0.77,ppl 2.1)。阶段二是 SFT(监督微调):在预训练基础上,用"题目→解法"数据,教它"看题→写解法"。

完整链:预训练(会写)→ SFT(对齐:看题写解)→ RL(泛化:真解对)。本篇是 SFT,以及一路踩的坑和最后看清的一个根本问题——0.6B 的天花板。

1. SFT:warm start + 题解格式 + 答案-only loss

SFT 相比预训练,模型结构不变,改三件事:

- **warm start**:加载预训练 checkpoint(会写 C++),继续训,不是从头。
- **数据格式**:"题目 + 解法"对,用标记分隔(`### Problem: ... ### Solution: ...`),解法末尾加 `<eos>` (学会何时停止)。

- **loss 只在答案(solution)算**:题目部分 label 设 -100(不算 loss,只当条件);模型学的是"给定题目,生成解法"。

机制还是 next-token + CE(和预训练一样),只是数据结构化(题+解)+ loss 只在答案。SFT loss 从预训练的 0.77 起点(warm start,已会写)降到 ~0.5(学会看题写解)。

SFT 快得惊人(~1 小时):数据量小(~1000 万 token,预训练 14B 的千分之几)。因为 SFT 是"对齐"(激活预训练已有能力),不是"学能力"——小数据够。有论文(LeetCodeDataset)发现 **2.6K 高质量 SFT ≈ 110K 数据**,印证"SFT 质量 > 数量"。

2. 变长样本组 batch:一个 mask 就够

SFT 每个样本是"题+解"一组,长度不一。组 batch 要 padding 到最长。一个自然的问题:padding 要不要专门 mask?

结论:一个 loss mask 就够(题目=0, 解=1, padding=0)。不需要额外的 attention padding mask,因为:

- padding 加在**末尾**;
- 模型有 **causal mask**(每个 token 只看前面);
- 真 token(题+解)都在 padding 前面 → 被 causal 挡住,天然看不到后面的 padding;
- padding 自己的输出 loss mask=0 不算,无所谓。

所以 padding 末尾 + causal = 天然隔离,一个 loss mask 搞定。(padding mask 和 causal mask 本质一样:都是"不该看的地方设 0"。)

3. 数据清洗的反复踩坑

SFT 效果好坏,几乎全看数据质量。这一段是反复踩坑的过程,值得记录——每一步都是被"生成结果不对"逼着往回查。

第一坑:混入 Python(CodeContests 的 language 标错)

第一版 SFT 数据混了 Magicoder(标 cpp)+ CodeContests(筛 language==3 即 C++)。训完生成:第 1 题(找最大值)写对了,但 **factorial 蹦出 Python(for i in range(1,n))**、其他题复读。往回查:CodeContests 的 **language==3 标错了**——很多标成 C++ 的其实是 Python(`from collections import deque`、`n=int(input())`)。数据污染了模型。

第二坑:编译验证太严(误筛真 C++)

改用 g++ 编译验证(能编译=真 C++)。结果 **只 14% 通过(1745/12827)**,而且 CodeContests 几乎全军覆没。

看失败例子才发现两类:① CodeContests 确实很多是 Python(该筛,编译不过对)② **但 Magicoder 是真 C++,只是依赖特殊库(`#include <GL/glew.h>`)编译不过,被误筛**。编译验证太严:片段/缺库的真 C++ 也被筛掉。

权衡:启发式(排 Python + 确认 C++ 特征)

最终用**启发式筛**(不用严格编译):排 Python 特征(`def/print/range/input`)+ 确认 C++ 特征(`#include/cout/std/{}/;` ≥ 3)。既排除 Python 污染,又保留缺库的真 C++。

再加一个干净源 **AmareshHebbar/leetcode-codegen-cpp**(来自 doocs 社区题解库,人写的正确解,messages 格式:user=题, assistant=解)。

最终:**Magicoder 3567 + AmareshHebbar 2901 = 6468 条干净 C++ 题解**。

清洗的教训:① 别信元数据的语言标注(CodeContests 的 language 标错)——要验证 code 内容。② 编译验证最准但太严(片段/缺库的真代码被误筛),启发式是务实权衡。③ 数据质量 > 数量:脏的 12773 训出来只 1 题对,干净的 6468 明显好——但仍受限于下一节的天花板。

4. Raw 模型训完后的检查

预训练出 raw 模型(loss 0.77)后,做了两类检查:① 静态诊断(看模型内部学成什么样)② 生成测试(实际能不能用)。两个都有重要发现。

4.1 Token 覆盖诊断:哪些 token 训到了

全量扫 14B token,统计每个 token 的出现频率(=BP 更新次数),看词表覆盖:

类别	数量	占比
touched(出现过)	30245	94.5%
never(从没出现)	1761	5.5%
高频(≥ 1 万次)	27594	86%

never 的 1761 个是什么:① 特殊 token(`<pad>/<bos>/<unk>` ,预训练只用 eos 分隔,这几个没用到)② 生僻高位字节(`À Á Õ Ö ÷ ...ÿ` ,即 0x80-0xFF,C++ 里几乎不出现)③ **意外:带空格前缀的保护关键词(`Ġif Ġreturn Ġconst Ġstd`)也 never**——说明代码里这些关键词更常以别的形式(无前导空格)出现。

4.2 单字节兜底分析

byte-level BPE 里"没被 merge 的都退化成单字节"。统计:367 个单字节类 token,13 个 never(正是生僻高位字节)。**单字节总占 42.3% 的 token 出现**——大量内容(符号/空格/单字母)以单字节形式存在。这印证:**不是每个单字节都会被 touch**(常见字符高频,生僻字节 never),而"没见过的东西最终都落在单字节上"(兜底层)。

4.3 embedding 稀疏更新验证(第一性原理)

BP 对 embedding 是稀疏更新:只有出现过的 token 那一行有梯度、被更新;没出现的梯度恒 0、embedding 不变(数学必然)。验证——按频率分桶看 embedding 范数:

频率	token 数	平均范数
never(0)	1761	0.751
1~100	75	0.755
100~1万	2576	0.926
1万~100万	26283	1.023

频率-范数完美单调对应:never 的范数 $0.751 \approx$ 随机初始化理论值($0.02 \times \sqrt{d} \approx 0.716$);频率越高范数越偏离随机(高频 1.02)。**这定量验证了"出现才训"**——没出现的 embedding 还是随机初始值。**妙处**:判断 never 不需要存训练前的初始 embedding(没存)——因为初始化是已知分布 $N(0, 0.02^2)$,never 的 embedding 必然仍服从这个分布(0 次更新没动),检查"是否仍 $\approx$ 随机分布"即可。用"已知的初始分布"替代"没存的初始具体值"。

4.4 生成测试:发现 KV-cache 的 RoPE bug

静态诊断显示模型训得好(loss 0.77、覆盖 94.5%、embedding 正常),但一跑生成(KV-cache)就**全是复读**——这暴露了下一节的 bug。

5. KV-cache 的 RoPE 位置 bug:训练对、推理崩

预训练完 loss 0.77(很好),但用 KV-cache 快速生成时**全是复读**(`return return return...`)。loss 好+生成崩 = 不是模型问题,是生成代码(KV-cache)的 bug。

根因:增量生成时 RoPE 位置用了 0

训练(整序列 forward,不用 KV-cache):RoPE 对每个位置正确应用 $\rightarrow$ 位置对 $\rightarrow$ loss 好。

推理(KV-cache 增量,一个个吐 token): `apply_rope` 用 `cos/sin[:seq]`,增量时 `seq=1` $\rightarrow$ 取的是**位置 0** 的旋转 $\rightarrow$ 新 token(其实是第 n 个)被当"位置 0" $\rightarrow$ 位置全错 $\rightarrow$ 崩。

训练路径(整序列,位置对)和推理路径(KV-cache,位置 bug)是两条——训练对、推理错,所以"训好但生成崩"。

定量:为什么位置错就复读

设 head_dim=2, K 原始都 [1,0]。

- **位置对**: K 按位置 0/1/2 旋转 → [1,0]/[0.54,0.84]/[-0.42,0.91]; $Q(\text{位置3}) \cdot K = -0.99/-0.41/0.55$ (**不同**) → softmax 权重有区分(0.10/0.24/0.66) → 关注不同位置。

- **位置全错成 0**: K 全不旋转 = 全 [1,0]; $Q \cdot K = 1/1/1$ (**全相同**) → 权重均匀(0.33/0.33/0.33) → 对每个位置注意力一样 → **每步输出雷同** → **复读**。

位置错 → 旋转全 0 → $Q \cdot K$ 失去区分 → 权重退化成均匀 → 复读。

修复: apply_rope 加 offset 参数,增量生成时传当前位置(offset=n),取 $\cos/\sin[\text{offset}:\text{offset}+\text{seq}]$ = 真实位置的旋转。修完 KV-cache 生成正常(不复读)。

教训: "训练/推理路径不一致"是 LLM 的经典 bug——训练用整序列(位置天然对),推理用 KV-cache 增量(位置要手动传对)。loss 好不代表生成对,两条路径都要验证。

6. 验证:见过的题 vs 没见过的题

SFT 完(干净 6468 数据),验证分两种:训练集常见题型 vs 刻意编的没见过的题。对比暴露了 SFT 的本质。

见过的常见题:部分对

"数组求和" → 完全正确(读输入+累加+输出); "factorial" → 结构对(类+递归),就基例错(`return 0` 应 `return 1`); "最长子串/回文" → 结构像(双指针/滑窗的样子)但逻辑错。

基本会写像样 C++ 结构,简单题逻辑对,复杂题结构对逻辑错。比脏数据的"1 题对+蹦 Python"明显进步。

没见过的题:泛化很差(张冠李戴)

刻意编的新题(大概率不在训练集):

- "返回第二大元素" → 写成 **线段树**(答非所问)
- "过滤长度>5 的词" → 写成 **归并排序**(答非所问)
- "数 n 以内质数个数" → 写成 **数因子**(逻辑错)
- "反转句中词序" → 写成正负数 reorder(答非所问)

6 题基本全错,而且大量 **答非所问**——模型没读懂题,只抓几个词就套见过的模式。

见过 vs 没见过的鲜明对比 = SFT 记忆题型形式,不泛化。更深的问题:大量"答非所问"说明模型读不懂题(自然语言理解弱)——这不是数据脏(LeetCode 数据是对的),是模型能力:纯 C++ 预训练(几乎没有自然语言)→ 题目理解弱 → 抓词套模式 → 张冠李戴。

7. 迟到的教训:0.6B 的天花板

验证到这里,一个本该一开始就正视的问题浮出水面:0.6B 这个规模,解题能力的天花板极低。

模型	预训练数据	HumanEval pass@1
Qwen2-0.5B(业界最强 0.5B)	5.5 万亿 token	~17%
Qwen2.5-Coder-1.5B	5.5 万亿 token	~44%
Qwen2.5-Coder-7B	5.5 万亿 token	~85%
本项目 0.6B	14B token(少 ~400 倍)	远低于 17%

业界最强的 0.5B(5.5 万亿 token 训的)HumanEval 才 17%(164 题只解对约 28 道)。本项目 0.6B 用 14B token(少 400 倍),天花板更低——"张冠李戴、读不懂题"是必然,不是 SFT/RL 做错。

最重要的教训:能力的地基是预训练。 0.6B + 14B token 的预训练太弱(读题弱、逻辑弱),SFT/RL 都救不了。而且——"策略太差,指望 RL 救,很难":RL 是在现有策略上试错优化,策略差到采样全错(没正样本),RL 学不动(这和 CarRacing 里"探索撞不到正样本就学不动"一模一样)。RL 放大好策略,救不了烂策略。

这个天花板本该在选 0.6B 时就正视:若目标是"学机制、走通全流程",0.6B 没问题(便宜、完整);若目标是"真解题效果",0.6B 一开始就不够,该用现成强 base。

8. 路线修正:从零训 vs 现成强 base

从零训小模型(本项目走的):预训练地基弱(14B token),天花板低。价值在**学机制**(懂了预训练/SFT/RoPE/KV-cache/GRPO 的底层),不在解题效果。

现成强 base + post-train(正确路线):下载 Qwen2.5-Coder / Gemma 等(已在几万亿 token 训好,HumanEval 44-85%),**预训练这一步"白嫖"**(大厂烧的几十万刀),自己只做 SFT + RL(小钱 ~\$1000)。站在强 base 上:SFT 有效、RL 有意义(好策略有正样本,提升大,如 DeepSeek-R1 靠 RL 把 AIME 从 15% 提到 78%)。

本项目的真正价值:从零手写走通预训练→SFT 全流程,推穿 $CE=\log|\text{vocab}|$ 、困惑度=等效候选数、RoPE 旋转/KV-cache、SFT 的 mask loss、GRPO 原理,踩透数据清洗/显存/速度/训练-推理不一致

的坑。这些底层理解,是"用现成 base + post-train"时的硬底子——知道每一步在干什么,而不是黑盒调 API。

9. 能力占比:预训练是地基

阶段	贡献(粗略)	作用	本项目的坑
预训练	~80-90%	会写代码/读题/算法知识(地基)	14B token 太弱(根源)
SFT	~5-10%	对齐格式(立竿见影但记忆)	数据清洗反复;泛化差
RL	~5-15%	泛化+精修(要好 base 才有效)	0.6B 策略差,RL 难救

关键认识:SFT 立竿见影(对齐),容易让人以为"SFT 才是关键、RL 有限"。但这是差 base 上的错觉——强 base + 好 SFT 上,RL 提升巨大(R1)。真正的地基永远是预训练;SFT 对齐、RL 泛化,都是在地基上锦上添花。本项目 0.6B 弱,是地基(预训练)弱,不是 SFT/RL 的锅。

下一步

放弃"从零训 0.6B solve LeetCode"(天花板注定),转向**现成强 base + post-train(SFT+RL)**。目标落到 **Google Gemma 4 Developer Agent 竞赛**——post-train Gemma 4 成自主 coding agent(SFT+RL,离线,agentic),正是本项目积累(SFT/RL 代码、数据清洗、底层数学)的用武之地,且路线正确(强 base + post-train)。

— 阶段二完。教训:预训练是地基,0.6B 天花板注定;转向强 base + post-train。 —